

Python One-Stop Regular Problems and Solutions

Collections, iterators/generators, asyncio/threading, caching, and core DSA/Algo in one interview-focused guide.

Section A: Collections and Built-ins (P1-P10)

P1. Two Sum

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
    return [-1, -1]
```

P2. Group Anagrams

```
from collections import defaultdict

def group_anagrams(words):
    g = defaultdict(list)
    for w in words:
        g[''.join(sorted(w))].append(w)
    return list(g.values())
```

P3. Top K Frequent

```
from collections import Counter
import heapq

def top_k(nums, k):
    return [x for x, _ in Counter(nums).most_common(k)]
```

P4. Merge Intervals

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    out = []
    for s, e in intervals:
        if not out or out[-1][1] < s:
            out.append([s, e])
        else:
            out[-1][1] = max(out[-1][1], e)
    return out
```

P5. Sliding Window Maximum

```
from collections import deque

def max_sliding_window(nums, k):
    dq, ans = deque(), []
    for i, x in enumerate(nums):
        while dq and dq[0] <= i - k:
            dq.popleft()
        while dq and nums[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            ans.append(nums[dq[0]])
    return ans
```

P6. LRU cache via functools

```
from functools import lru_cache

@lru_cache(maxsize=1024)
def parse_user(uid):
    return {'id': uid}
```

P7. Counter patterns

```
from collections import Counter
c = Counter('abca')
```

P8. defaultdict patterns

```
from collections import defaultdict
g = defaultdict(list)
```

P9. heapq min-heap

```
import heapq
h = [3,1,2]
heapq.heapify(h)
```

P10. deque as queue

```
from collections import deque
q = deque([1,2,3]); q.popleft()
```

Section B: Iteration, Generators, and Functional (P11-P16)

P11. Flatten nested lists

```
def flatten(nested):  
    return [x for row in nested for x in row]
```

P12. Generator pipeline

```
def gen(nums):  
    for n in nums:  
        if n % 2 == 0:  
            yield n * n
```

P13. itertools combinations

```
from itertools import combinations  
list(combinations([1,2,3], 2))
```

P14. map/filter/reduce practical usage

```
from functools import reduce  
  
nums = [1, 2, 3, 4]  
evens_sq = list(map(lambda x: x * x, filter(lambda x: x % 2 == 0, nums)))  
prod = reduce(lambda a, b: a * b, nums, 1)
```

P15. sorting with key and stability

```
people = [{"name": "a", "age": 30}, {"name": "b", "age": 30}, {"name": "c", "age": 25}]  
people.sort(key=lambda p: p["age"]) # stable sort keeps a before b
```

P16. dataclass for model objects

```
from dataclasses import dataclass  
  
@dataclass(order=True)  
class Task:  
    priority: int  
    title: str
```

Section C: Threads, Asyncio, and Concurrency (P17-P24)

P17. ThreadPoolExecutor for I/O

```
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=20) as ex:
    futures = [ex.submit(lambda x=i: x*x) for i in range(10)]
    out = [f.result() for f in futures]
```

P18. asyncio gather fan-out

```
import asyncio

async def work(i):
    await asyncio.sleep(0.01)
    return i

async def run():
    return await asyncio.gather(*(work(i) for i in range(5)))
```

P19. asyncio timeout

```
await asyncio.wait_for(coro(), timeout=0.5)
```

P20. lock-protected shared state

```
import threading
lock = threading.Lock()
```

P21. queue-based producer-consumer

```
import queue
q = queue.Queue(maxsize=100)
```

P22. multiprocessing for CPU-bound tasks

```
from concurrent.futures import ProcessPoolExecutor

def sq(x):
    return x * x

with ProcessPoolExecutor() as ex:
    out = list(ex.map(sq, [1, 2, 3, 4]))
```

P23. cancellation and graceful shutdown

```
import asyncio
```

```

async def worker():
    try:
        while True:
            await asyncio.sleep(0.1)
    except asyncio.CancelledError:
        # cleanup
        raise

```

P24. idempotent retries with backoff

```

import time

def call_with_retry(fn, attempts=3):
    delay = 0.1
    for i in range(attempts):
        try:
            return fn()
        except Exception:
            if i == attempts - 1:
                raise
            time.sleep(delay)
            delay *= 2

```

Section D: Cache and Reliability (P25-P30)

P25. TTL cache minimal pattern

```

import time

class TTLCache:
    def __init__(self):
        self.m = {}
    def get(self, k, loader, ttl=60):
        v = self.m.get(k)
        now = time.time()
        if v and v[1] > now:
            return v[0]
        val = loader(k)
        self.m[k] = (val, now + ttl)
        return val

```

P26. Cache stampede prevention (per-key lock)

```

import threading
from collections import defaultdict

locks = defaultdict(threading.Lock)
cache = {}

```

```
def get_or_load(key, loader):
    if key in cache:
        return cache[key]
    with locks[key]:
        if key not in cache:
            cache[key] = loader(key)
        return cache[key]
```

P27. Negative caching with short TTL

```
# store (value_or_none, expiry) even for misses to reduce repeated expensive lookups
```

P28. Eviction policy tradeoffs (LRU/LFU/FIFO)

Use LRU for temporal locality, LFU for hot-key stability, FIFO for simplicity.

P29. Serialization choices (json/msgpack/pickle caution)

Use `json` for interoperability, `msgpack` for compactness, avoid untrusted `pickle` payloads.

P30. Cache invalidation strategy and versioning

Use versioned keys (`user:42:v17`) or publish invalidation events on writes.

Section E: DSA/Algo Core (P31-P45)

P31. Binary search

```
def binary_search(a, target):
    l, r = 0, len(a) - 1
    while l <= r:
        m = (l + r) // 2
        if a[m] == target:
            return m
        if a[m] < target:
            l = m + 1
        else:
            r = m - 1
    return -1
```

P32. BFS shortest path

```
from collections import deque

def shortest_path(g, src, dst):
    dist = [-1] * len(g)
    q = deque([src])
    dist[src] = 0
```

```

while q:
    u = q.popleft()
    if u == dst:
        return dist[u]
    for v in g[u]:
        if dist[v] == -1:
            dist[v] = dist[u] + 1
            q.append(v)
return -1

```

P33. DFS cycle detection

```

def has_cycle(g):
    state = [0] * len(g)

    def dfs(u):
        if state[u] == 1:
            return True
        if state[u] == 2:
            return False
        state[u] = 1
        for v in g[u]:
            if dfs(v):
                return True
        state[u] = 2
        return False

    return any(dfs(i) for i in range(len(g)))

```

P34. Topological sort

```

from collections import deque

def topo_sort(n, edges):
    g = [[] for _ in range(n)]
    indeg = [0] * n
    for u, v in edges:
        g[u].append(v)
        indeg[v] += 1
    q = deque([i for i in range(n) if indeg[i] == 0])
    out = []
    while q:
        u = q.popleft()
        out.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return out if len(out) == n else []

```

P35. Dijkstra

```
import heapq

def dijkstra(g, src):
    inf = 10**18
    dist = [inf] * len(g)
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d != dist[u]:
            continue
        for v, w in g[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq, (nd, v))
    return dist
```

P36. Union-Find

```
class DSU:
    def __init__(self, n):
        self.p = list(range(n))
        self.r = [0] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.r[ra] < self.r[rb]:
            self.p[ra] = rb
        elif self.r[ra] > self.r[rb]:
            self.p[rb] = ra
        else:
            self.p[rb] = ra
            self.r[ra] += 1
        return True
```

P37. Kadane

```
def max_subarray(nums):
    best = cur = nums[0]
```



```

for x in nums[1:]:
    cur = max(x, cur + x)
    best = max(best, cur)
return best

```

P38. Prefix sum subarray-k

```

from collections import defaultdict

def subarray_sum(nums, k):
    cnt = defaultdict(int)
    cnt[0] = 1
    s = ans = 0
    for x in nums:
        s += x
        ans += cnt[s - k]
        cnt[s] += 1
    return ans

```

P39. LIS $O(n \log n)$

```

from bisect import bisect_left

def lis(nums):
    tails = []
    for x in nums:
        i = bisect_left(tails, x)
        if i == len(tails):
            tails.append(x)
        else:
            tails[i] = x
    return len(tails)

```

P40. Coin change DP

```

def coin_change(coins, amount):
    inf = amount + 1
    dp = [0] + [inf] * amount
    for a in range(1, amount + 1):
        dp[a] = min((dp[a - c] + 1 for c in coins if a - c >= 0), default=inf)
    return -1 if dp[amount] == inf else dp[amount]

```

P41. Knapsack DP

```

def knapsack(weights, values, cap):
    dp = [0] * (cap + 1)
    for w, v in zip(weights, values):

```

```

    for c in range(cap, w - 1, -1):
        dp[c] = max(dp[c], dp[c - w] + v)
    return dp[cap]

```

P42. Trie

```

class TrieNode:
    def __init__(self):
        self.n = {}
        self.end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, s):
        cur = self.root
        for ch in s:
            cur = cur.n.setdefault(ch, TrieNode())
        cur.end = True

    def search(self, s):
        cur = self.root
        for ch in s:
            if ch not in cur.n:
                return False
            cur = cur.n[ch]
        return cur.end

```

P43. Backtracking subsets

```

def subsets(nums):
    out = []
    cur = []

    def dfs(i):
        if i == len(nums):
            out.append(cur.copy())
            return
        dfs(i + 1)
        cur.append(nums[i])
        dfs(i + 1)
        cur.pop()

    dfs(0)
    return out

```

P44. Backtracking permutations

```
def permutations(nums):
    out = []
    used = [False] * len(nums)
    cur = []

    def dfs():
        if len(cur) == len(nums):
            out.append(cur.copy())
            return
        for i, x in enumerate(nums):
            if used[i]:
                continue
            used[i] = True
            cur.append(x)
            dfs()
            cur.pop()
            used[i] = False

    dfs()
    return out
```

P45. Interval scheduling

```
def max_non_overlapping(intervals):
    intervals.sort(key=lambda x: x[1])
    ans = 0
    end = float('-inf')
    for s, e in intervals:
        if s >= end:
            ans += 1
            end = e
    return ans
```

Section F: Advanced Production Patterns (P46-P55)

P46. Async fan-out with bounded semaphore

```
import asyncio

sem = asyncio.Semaphore(20)

async def call_limited(fn, *args):
    async with sem:
        return await fn(*args)
```

P47. Structured logging pattern

Use JSON logs with request_id, user_id, latency_ms, and error_code.

P48. Resilient HTTP client

Timeouts + retries + jitter + status-based retry policy.

P49. Idempotency key for write APIs

Persist key and response digest to prevent duplicate side effects.

P50. Graceful service shutdown

Drain queue, stop intake, await in-flight tasks, close resources.

P51. Data validation boundary

Validate at edges (API/input), keep core domain assumptions strict.

P52. Backpressure strategy

Bounded queues and explicit overload response (429 /drop policy).

P53. Cache warmup and refresh-ahead

Preload top keys and refresh before TTL expiry for stable latency.

P54. Metrics baseline

Track rate, errors, p95, p99, queue depth, cache hit ratio.

P55. Testing strategy

Unit tests + property checks + load test + fault injection.

Suggested Practice Order

1. P1-P10 for collection fluency.
2. P11-P16 for iterator/generator maturity.
3. P17-P30 for concurrency and caching correctness.
4. P31-P45 for DSA pattern depth.
5. P46-P55 for production-readiness discussions.

Internet-Backed Reference Pointers

1. Python docs (language reference): <https://docs.python.org/3/reference/>
2. Python stdlib docs: <https://docs.python.org/3/library/>
3. asyncio docs: <https://docs.python.org/3/library/asyncio.html>
4. concurrent.futures docs: <https://docs.python.org/3/library/concurrent.futures.html>
5. collections docs: <https://docs.python.org/3/library/collections.html>
6. functools docs (lru_cache): <https://docs.python.org/3/library/functools.html>
7. heapq docs: <https://docs.python.org/3/library/heapq.html>